МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования «Северо-Кавказский федеральный университет»

Отчет по лабораторной работе № 1

«Основы Python и работа в VS Code и Zed»

по дисциплине «Программирование на языке Python»

Выполнил студент группы ЭНЭ-б-о-25-1
Величко С.Р. « »_____20__г.
Подпись студента_____
Работа защищена « »_____20__г.
Проверила Хацукова А.И. _____

Ставрополь 2026

Цель работы: научиться проверять готовность среды Python, создавать и запускать программы в VS Code и Zed, использовать переменные, арифметические операции, консольный ввод и форматированный вывод, а также исследовать выполнение программы с помощью отладчика.В

ОС: Windows 11
Версия Python: 3.15.2
Ссылка на GitHub: https://github.com/N-cat-210/python-lab-01

Задание 1: Знакомство с пользователем - user.py

Запросить имя, возраст, город, а затем вывести три строки с подписями. Имя и город - непустые строки, возраст - целое число.

```python
n = input("Введите имя: ")
a = int(input("Введите возраст: "))
c = input("Введите название города: ")
print(f"Имя: {n}")
print(f"Возраст: {a}")
print(f"Город: {c}")
```

Рисунок 1. Листинг программы user.py

```
Введите имя: Энтони
Введите возраст: 23
Введите название города: Оттава
Имя: Энтони
Возраст: 23
Город: Оттава
```

Рисунок 2. Результат выполнения

Преобразование возраста в число не обязательно, поскольку над значением возраста в дальнейшем не выполняются никакие операции. Необходимым оно станет только тогда.

Задание 2. Арифметический пример - arithmetic.py.

Предложить пользователю вычислить 4 * 100 - 54. Программа должна вычислить правильный ответ и вывести его вместе с ответом пользователя.

```python
u_r = input("Чему равно 4 * 100 - 54?: ")
r = 4*100-54
print(f"Ваш ответ: {u_r}")
print(f"Верный ответ: {r}")
```

Рисунок 3. Листинг программы arithmetic.py

```
Чему равно 4 * 100 - 54?: Крекекры
Ваш ответ: Крекекры
Верный ответ: 346
```

Рисунок 4. Результат

Преобразование через int() не обязательно, поскольку над введённым значением в дальнейшем не выполняются никакие операции.

Задание 3. Отношение сумм – numbers.py

Ввести четыре вещественных числа a, b, c, d каждое с новой строки. Вычислить (a+b)/(c+d). Вывести результат с двумя знаками после десятичной точки.

numbers.py > ...

```python
print("Введите четыре вещественных числа, каждое с новой строки:")
a = int(input("a: "))
b = int(input("b: "))
c = int(input("c: "))
d = int(input("d: "))
res = round((a+b)/(c+d), 2)
print(f"Результат: {res}")
```

Рисунок 5. Листинг программы numbers.py

```
Введите четыре вещественных числа, каждое с новой строки:
a: 67
b: 5
c: 145
d: 89
Результат: 0.31
```

Рисунок 6. Результат

Индивидуальное задание 1: Вариант 4.

4	Вещественные `a`, `b`, причём `b ≠ 0`	Сумму, разность `a − b`, произведение, частное `a / b`	`6. 2 → 8.00; 4.00; 12.00; 3.00`

Рисунок 7. Условия 4 варианта

individual_01.py > ...

```python
print("Введите числа a и b")
a = int(input("a: "))
b = int(input("b: "))
if  b == 0:
    print("Ошибка! b не может быть равно 0")
else:
    Ans1 = round(a-b, 2)
    Ans2 = round(a*b, 2)
    Ans3 = round(a/b, 2)
    print(f"Результаты: {Ans1}; {Ans2}; {Ans3}")
```

Рисунок 8. Код

```
Введите числа a и b
a: 56
b: 0
Ошибка! b не может быть равно 0
```

Рисунок 9. Результат при b=0

```
Введите числа a и b
a: 77
b: 5
Результаты: 72; 385; 15.4
```

Рисунок 10. Результат при значении b>0

Индивидуальное задание 2: Вариант 4.

Введите цифры десятков и единиц положительных двузначных чисел в порядке a2, a1, b2, b1. Гарантируется, что сумма чисел двузначная. Найдите цифры десятков и единиц суммы, не составляя полные числа-слагаемые и полную сумму.

individual_02.py

```python
a2 = int(input("Введите a2: "))
a1 = int(input("Введите a1: "))
b2 = int(input("Введите b2: "))
b1 = int(input("Введите b1: "))
s = a1 + b1
s1 = s % 10
c = s // 10
s2 = a2 + b2 + c
if (s2 >= 10):
    print ("Ошибка! Сумма a2 и b2 слишком большая")
else:
    print(f"Цифры суммы: {s2}; {s1}")
```

Рисунок 11. Код в Zed

```
Введите a2: 9
Введите a1: 9
Введите b2: 9
Введите b1: 9
Ошибка! Сумма a2 и b2 слишком большая
```

Рисунок 12. Результат при недвузначной сумме

```
Введите a2: 5
Введите a1: 4
Введите b2: 3
Введите b1: 2
Цифры суммы: 8; 6
```

Рисунок 13. Результат в ином случае

Ответы на контрольные вопросы:

1. Редактор текста (например, Zed) предназначен только для написания и форматирования кода. Интерпретатор (Python) — это программа, которая выполняет этот код, переводя его в машинные команды.
    - В VS Code поддержка Python обеспечивается официальным расширением Python (от Microsoft), которое ищет интерпретаторы на диске.
    - В Zed поддержка идет через встроенный протокол LSP (Language Server Protocol) и расширение для Python, взаимодействующее с сервером pyright или ruff.
2. Узнать версию и путь можно через терминал командами python --version и where python (Windows) или which python (macOS/Linux), либо внутри скрипта через import sys; print(sys.version, sys.executable).
    - Несколько интерпретаторов на компьютере может быть из-за установки разных версий (например, Python 3.10 и 3.12), работы разных программ (Anaconda, базы данных) или использования изолированных сред разработки.
1. Папка .venv (виртуальное окружение) создается для изоляции зависимостей конкретного проекта.
    - Окружение проекта содержит только те библиотеки и ту версию Python, которые нужны для текущей программы.
    - Глобальная среда едина для всей системы. Если ставить все пакеты туда, возникнет конфликт версий (одному проекту нужна старая версия библиотеки, другому — новая).
3. Выбор окружения .venv:
    - В VS Code: Нажать Ctrl+Shift+P (или Cmd+Shift+P), ввести Python: Select Interpreter и указать путь к файлу python внутри папки .venv.
    - В Zed: Zed обычно автоматически подхватывает .venv, если открыть папку проекта. Также можно указать путь в настройках settings.json проекта.
    - Проверка в терминале: При активации среды в начале строки терминала появляется имя окружения в скобках: (.venv). Команда which python / where python покажет путь, ведущий внутрь папки проекта.
4. REPL (интерактивный режим) выполняет каждую строчку кода сразу после нажатия Enter — это удобно для быстрых тестов. Запуск .py файла выполняет весь написанный скрипт от начала до конца.
    - Приглашение “>>>” означает, что интерпретатор Python находится в интерактивном режиме и готов принимать команды.

6: Присваивание и типы данных (int, float, str)

• Присваивание (=) — операция связи имени переменной с объектом в памяти компьютера (создание ссылки).
• int — целые числа произвольной точности, ограниченные только доступной RAM (например, x = 100_000).
• float — вещественные числа с плавающей точкой стандарта IEEE 754, имеющие погрешность при хранении двоичных дробей (например, y = 0.1).
• str — неизменяемый последовательный тип данных для хранения текста в кодировке Unicode (например, z = "Код").

7: Работа с input(), int() и float()

• input() — функция, которая всегда возвращает прочитанные данные в виде строки (str), даже если введены цифры.
• int() / float() — функции явного приведения типов, необходимые для превращения текстовых цифр в полноценные числа.
• Зачем нужны: Без них невозможно совершать математические операции над вводом пользователя — вместо сложения чисел произойдет склеивание строк ("2" + "3" = "23").

8: Операторы /, // и %

• / — обычное деление, результатом которого всегда является вещественное число (float).
• // — деление нацело с округлением результата до ближайшего меньшего целого числа (к минус бесконечности).
• % — остаток от целочисленного деления, знак которого всегда совпадает со знаком делителя.
• Вычисление -7 // 4: Результат точного деления -1.75. Округление вниз (в меньшую сторону) дает -2.

9: Скобки в выражении (a + b) / (c + d)

• Влияние скобок: Они принудительно устанавливают наивысший приоритет для операций сложения.
• Логика вычисления: Сначала параллельно вычисляются суммы внутри скобок (a + b) и (c + d), и только после этого первый результат делится на второй.
• Без скобок: Выражение a + b / c + d сначала выполнило бы деление b / c, а затем прибавило бы к нему a и d.

10: Устройство f-строк и формат .2f

• f-строка — это литерал форматирования (начинается с префикса f"..."), который позволяет внедрять переменные и выражения прямо внутрь текста в фигурных скобках {}. Вычисляется динамически во время работы программы.
• Формат .2f — спецификатор вывода вещественного числа, который округляет его и жестко фиксирует ровно два знака после запятой.
• Влияние на переменную: Данный формат меняет только отображение числа на экране (превращая его в строку), сама исходная переменная в памяти не изменяется.

12: Назначение import math и единицы тригонометрических функций

• Назначение import math: Подключает встроенный модуль math, который предоставляет доступ к расширенным математическим функциям (корень, логарифмы, округления) и константам (pi, e).
• Единицы углов: Во всех тригонометрических функциях модуля math (например, math.sin(), math.cos()) углы задаются исключительно в радианах.
• Перевод единиц: Для корректной работы с градусами их необходимо предварительно перевести в радианы с помощью функции math.radians().

13: Точка останова и отладка в Zed

• Точка останова (breakpoint): Строковая метка в коде, на которой интерпретатор временно приостанавливает выполнение программы в режиме отладки для анализа состояния памяти.
• Запуск отладчика: Реализуется кликом слева от номера строки для установки точки и запуском сессии дебага через панель команд редактора.
• Просмотр переменных и шаг: Значение переменной проверяется наведением курсора или в специальном боковом окне отладки. Выполнение следующей строки осуществляется с помощью команды пошагового обхода (Step Over).

14: Наборы данных для проверки индивидуальных заданий

• Стандартные (позитивные) данные: Наборы корректных значений для проверки выполнения базовой математической или логической цепочки программы в штатном режиме.
• Граничные значения: Экстремальные точки диапазонов (нули, отрицательные границы, пустые строки), необходимые для проверки устойчивости алгоритма на краях условий.
• Негативные данные: Заведомо некорректные входные данные, используемые для проверки систем обработки ошибок и предотвращения критического падения программы.

15: Правила для имен переменных

• Основные правила: Имя может содержать латинские буквы, цифры и знаки подчеркивания. Оно не может начинаться с цифры и совпадать с ключевыми словами языка (if, while).
• Различие age и Age: Это разные переменные. Python строго регистрозависим.
• Запрет на input: Слово input — встроенная функция. Переменная с таким именем перекроет функцию, сделав ввод данных невозможным.

16: Динамическая типизация и type()

• Динамическая типизация: Тип переменной не заявляется заранее. Он определяется автоматически в момент присваивания значения и может меняться.

• Узнать тип: Функция type(переменная) возвращает класс объекта (например, <class 'int'>).

17: Различия "2" + "3" и int("2") + int("3")

• Выражение "2" + "3": Конкатенация (склеивание) строк. Результат: "23" (тип str).
• Выражение int("2") + int("3"): Математическое сложение целых чисел. Результат: 5 (тип int).

18: Чтение восьми координат через input()

• Реализация: Требуется вызвать функцию input() восемь раз подряд. Каждая координата должна вводиться с новой строки (через Enter) и сразу приводиться к числу: x1 = float(input()).

19: Ошибки приведения в float() и int()

• float("2,5"): Ошибка ValueError. Разделителем дроби в Python может быть только точка.
• int("2.5"): Ошибка ValueError. Функция ожидает только целое число без точек.
• Формат: Для float() нужна точка ("2.5"), для int() — отсутствие любых знаков разделения ("2").

20: Погрешность 0.1 + 0.2 и формат .2f

• Причина 0.30000000000000004: Ограничение двоичной системы компьютера, которая не может точно представить бесконечные дроби 0.1 и 0.2 в памяти.
• Влияние .2f: Изменяет только внешнее отображение числа при выводе. Значение самой переменной в памяти не меняется.

21: Порядок операций и выражения со степенью

• Порядок: Скобки -> Возведение в степень (**) -> Умножение/Деление (*, /, //, %) -> Сложение/Вычитание (+, -).
• -2 ** 2: Результат -4. Степень приоритетнее унарного минуса: -(2 ** 2).
• (-2) ** 2: Результат 4. Скобки заставляют возвести в квадрат отрицательное число.

• 2 ** -1: Результат 0.5. Возведение в отрицательную степень (\(2^{-1} = 1/2\)).

22: Выделение цифр трехзначного числа и перенос

• Выделение цифр для числа n:
  • Сотни: n // 100
  • Десятки: (n // 10) % 10
  • Единицы: n % 10
• Учет переноса: Сумма разрядов единиц делится нацело на 10: перенос = (ед1 + ед2) // 10. Полученный результат (0 или 1) прибавляется к сумме десятков.

23: Движение по кольцу и переход через полночь

• Реализация: Используется оператор остатка от деления (%).
• Формула: новая_позиция = (текущая_позиция + шаг) % размер_кольца.
• Пример (время): Если на часах 22 и прошло 5 часов: (22 + 5) % 24 = 3 (3 часа ночи).

24: Различия abs(), sqrt(), floor(), ceil()

• abs(x): Возвращает модуль (абсолютное значение) числа.
• math.sqrt(x): Вычисляет квадратный корень. Ограничение: аргумент x должен быть строго неотрицательным (\(x \geq 0\)).
• math.floor(x): Округляет число до ближайшего целого вниз (в меньшую сторону).
• math.ceil(x): Округляет число до ближайшего целого вверх (в большую сторону).

25: Чтение Traceback и виды ошибок

• Поиск в сообщении: Имя файла и строка указаны в блоке File "...", line X. Причина сбоя — в самой последней строке.
• SyntaxError: Нарушение правил грамматики языка (забытая скобка, двоеточие). Код не запускается.
• ValueError: Тип данных верный, но значение некорректно (например, int("текст") или math.sqrt(-1)).
• ZeroDivisionError: Попытка деления числа на ноль.